

### A. Description & Requirements

This artifact appendix provides details on how to reproduce the results presented in OptiMix. OptiMix leverages two types of environments to evaluate its methodologies: (1) an analytical setting and (2) a simulation setting. In the analytical scenario, each technique is evaluated using datasets from Nym or RIPE, assessing the effectiveness of OptiMix methods in reducing latency while quantifying the associated anonymity loss. The methods evaluated include LONA (node adjustments), multiple routing strategies (LAR, GWR, GPR, and SSR), LBA (load balancing), and CRG (cover traffic generation). In contrast, the simulation setting—implemented in *Python* using the *SimPy* library—models the full end-to-end flow of communication in mixnets. Within the simulation, clients generate messages that are forwarded through mixnodes according to OptiMix routing strategies. Each mixnode applies mixing processes to anonymize and forward packets, with delays modeled using empirical measurements from Nym or RIPE to capture realistic inter-node transmission latency.

The artifact comprises approximately 20K lines of Python code and reproduces all results presented in the main body of the paper.<sup>1</sup>

1) *How to access*: The artifact repository is available via a persistent DOI: <https://doi.org/10.5281/zenodo.17054156>, and also on GitHub: <https://github.com/OptiMixnet/OptiMix>.

2) *Hardware dependencies*: Note that the artifact includes precisely the same configurations and settings presented in OptiMix. The only difference is that the number of iterations has been scaled down so that the artifact can run on standard systems with 16 GB RAM and 50 GB of disk space.

The estimated runtimes are based on executing the artifact on a system with the following specifications:

- 64-bit CPU: Intel® Core™ i7-10850H @ 2.70 GHz (1 core used)
- Physical Memory (RAM): 16 GB

3) *Software dependencies*: Additionally, the server must have Python 3.8.10 installed. Before running the experiments, it is necessary to verify that the `dependencies.txt` file is satisfied. We emphasize the importance of adhering to the specified hardware and software dependencies to ensure that the experiments complete within the expected time limits.

4) *Benchmarks*: OptiMix relies on latency and geographical data from two datasets: (i) *Nym*, which captures latency between nodes in the deployed Nym mixnet, and (ii) *RIPE*, which provides latency measurements among globally distributed nodes for broader network topologies. Both datasets are included in the repository.

<sup>1</sup>As the paper was accepted at the time of artifact submission, the AEC reviewed and validated all results reported in the main body of the camera-ready version.

### B. Artifact Installation & Configuration

After setting up a system with Python version 3.8.10, one can execute the artifact by first installing the required dependencies using the script specified in `dependencies.txt`, which is provided in the repository bundled with the code. Upon installing the dependencies, running `Main.py` with the arguments explained below will generate the results, which will be saved in the `Figures` or `Tables` directories.

### C. Major Claims

The major claims of OptiMix are summarized as follows:

- (C1): LATENCY PATTERN  
Fig. 2 illustrates the latency incurred in mixnets when applying different routing strategies. As the value of  $\tau$  increases, latency also increases across all strategies. Overall, LAR, GWR, and GPR exhibit comparable latency, while SSR consistently incurs the highest latency.
- (C2): ENTROPY PATTERN  
Fig. 3 shows that increasing  $\tau$  results in higher entropy across all routing strategies. In particular, LAR exhibits very low entropy for  $\tau \leq 0.6$ , while SSR consistently achieves the highest entropy overall.
- (C3): IMPACT OF LOAD BALANCING  
Fig. 4 evaluates the effect of applying the load balancing algorithm (LBA) to the routing strategies. It demonstrates that both latency and entropy increase across all values of  $\tau$  for cascade and stratified topologies—compared to the results in Figs. 2 and 3.
- (C4): OVERALL PERFORMANCE  
Fig. 5 highlights that when OptiMix is applied to cascade and stratified topologies, with or without LBA, the overall performance—measured as the ratio  $\text{Entropy/Latency}$ —is maximized when using either GPR or GWR strategies across all values of  $\tau$ .
- (C5): MIXNODE ADVERSARY  
Fig. 8 presents the fraction of fully compromised paths (FCP) under an adversary controlling a subset of mixnodes. The FCP is shown as a function of  $\tau$  and the adversarial budget  $\frac{f}{N}$ , where  $f$  denotes the number of compromised nodes. The results indicate that increasing  $\tau$  or decreasing  $\frac{f}{N}$  reduces the FCP.
- (C6): COVER ROUTING  
Fig. 9 demonstrates that applying the cover routing generation (CRG) method—especially with a slight increase in the cost parameter  $\theta$ —results in higher entropy and lower FCP across both stratified and cascade topologies.

### D. Evaluation

This section details the set of experiments conducted to support the main claims presented in the artifact appendix. Executing these experiments produces results stored in the `Results`, `Figures`, or `Tables` directories. Additionally, we explain how to run scripts to regenerate specific results corresponding to figures or tables presented in the main body of the paper.

1) *Experiment ( $E_0$ ) [Setup] ( $\leq 5$  min)*: This experiment prepares all required datasets and auxiliary files. It must be run once before executing any other experiment.

[Preparation and Execution] Run the following command with the `Input` argument set to 0:

```
python3 Main.py
```

[Results] The required files will be generated and stored in the `Results` folder or the root directory of the artifact.

2) *Experiment ( $E_1$ ) [Figures 2–5] ( $\leq 5$  min)*: This experiment supports claims **C1–C4** by generating baseline latency and entropy results.

[Preparation and Execution] Run the following command with the `Input` argument set to 1:

```
python3 Main.py
```

[Results] The results will be saved in the `Figures` folder as: Fig. 2a–2d, Fig. 3a–3d, Fig. 4a–4d, and Fig. 5a–5d.

[Note] You may encounter the following warnings during execution, which are safe to ignore:

```
RuntimeWarning: Mean of empty slice.
  out=out, **kwargs

invalid value encountered in scalar divide
  ret = ret.dtype.type(ret / rcount)
```

3) *Experiment ( $E_2$ ) [Figure 8] ( $\leq 5$  min)*: This experiment evaluates the impact of mixnode adversaries, supporting claim **C5**.

[Preparation and Execution] Run the following command with the `Input` argument set to 2:

```
python3 Main.py
```

[Results] The output will include Fig. 8a–8d, saved in the `Figures` folder.

4) *Experiment ( $E_3$ ) [Figure 9] ( $\leq 5$  min)*: This experiment evaluates the cover routing strategy and supports claim **C6**.

[Preparation and Execution] Run the following command with the `Input` argument set to 3:

```
python3 Main.py
```

[Results] The results will be saved in the `Figures` folder as Fig. 9a–9d.

5) *Experiment ( $E^*$ ) [All Others] ( $\leq 40$  min)*: This experiment allows the user to generate any figure or table shown in the main body of the paper, regardless of whether it is linked to a claim.

[Preparation and Execution] Refer to Table I below for the corresponding `Input` argument value and run:

```
python3 Main.py
```

[Results] Figures will be saved in the `Figures` folder. Tables will be saved in the `Tables` folder as PDF files.

TABLE I  
MAPPING OF INPUT ARGUMENTS TO SPECIFIC FIGURES AND TABLES.

| Experiment | Input | Experiment | Input |
|------------|-------|------------|-------|
| Fig. 2     | 22    | Fig. 3     | 33    |
| Fig. 4     | 44    | Fig. 5     | 55    |
| Fig. 6     | 66    | Fig. 7     | 77    |
| Fig. 8     | 88    | Fig. 9     | 99    |
| Tab. 1     | 100   | Tab. 2     | 200   |
| Tab. 3     | 300   | —          | —     |

However, in cases where a LaTeX installation is not available, table data should be printed directly in the terminal.

[Note] You may encounter the following warnings while running experiments related to Fig. 2–5 or Tab. 3. These warnings do not affect the correctness of the results and can be safely ignored:

```
RuntimeWarning: Mean of empty slice.
  out=out, **kwargs

invalid value encountered in scalar divide
  ret = ret.dtype.type(ret / rcount)
```

### E. Parameter Settings and Execution Time

The network parameter settings are largely consistent across all experiments and match those described in the evaluation setup (see page 7 of the main body). Accordingly, `config.py` is initialized to reflect these settings precisely. In each experiment, however, a single parameter is varied. For example, in Figs. 2–7 and Tabs. 1–2, the parameter  $\tau$  is varied; Fig. 8 varies the adversary’s budget  $\frac{f}{N}$ , and Fig. 9 varies the cost parameter  $\theta$ .

The artifact automatically applies the corresponding configuration changes when the appropriate input argument is provided. Therefore, manual modification of `config.py` is not required to reproduce the experiments.

The only difference between the artifact and the paper lies in the number of iterations used in certain experiments, which is set to 5 in the artifact to ensure that results are generated within a reasonable timeframe (each experiment completes in under 40 minutes). Users who wish to increase the number of iterations may do so manually in `config.py`.

Users may also modify `config.py` to explore experiments not explicitly evaluated in the paper. However, such modifications require a deeper understanding of mixnets, as these are complex systems where changes to one parameter may influence others. In many cases, specific parameter combinations can be incompatible or may significantly affect the results. Moreover, some parameters are initialized based on prior work, with default values selected to enable meaningful comparisons.

Nonetheless, the following parameters in `config.py` can be safely modified within specific intervals:

- `Iterations = 5` → Can be increased to improve accuracy; note that this change increases computational cost linearly.

- `Num_targets = 200` → Specifies the number of target messages in simulations; can be set to any integer in  $[20, 200]$ .
- `run_time = 0.5` → Defines the duration of each simulation time slot; can be set to any real value in  $[0.3, 1.0]$ .
- `delay1 = 0.05` → Represents the average delay imposed on each message upon entering mixnodes; can be set to any real value in  $[0.03, 0.08]$ .
- `e2e_delay` → Defines the end-to-end latency limit used in Tabs. 1 and 2; can be varied between 0.16 and 0.4.

Other parameters in the configuration should not be modified without sufficient expertise. We recommend that users contact the authors directly if such changes are necessary.